

# Diabetes Dataset: Cleaning, Imputation, Distance Metrics & Standardization

## Imports

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.metrics.pairwise import (
    euclidean_distances,
    manhattan_distances,
    cosine_distances
)
```

## Load Dataset

```
df = pd.read_csv("diabetes.csv")

# Clean column names
df.columns = df.columns.str.strip().str.lower()

print("Columns in Dataset:")
print(df.columns.tolist())

print("\nFirst 5 Rows:")
print(df.head())
```

## Validate Required Columns

```
columns = [
    'glucose_concentration',
    'diastolic_blood_pressure',
    'triceps_skin_thickness',
    'serum_insulin',
    'bmi'
]

missing = [col for col in columns if col not in df.columns]

if missing:
    print("ERROR!")
    print("These columns were not found:", missing)
```

```
print("\nAvailable columns are:")
print(df.columns.tolist())
```

## Handle Missing Values

Replace zero values (used as placeholders) with NaN, then impute using the median.

```
# Replace zero values with NaN
df[columns] = df[columns].replace(0, np.nan)

print("\nMissing Values:")
print(df.isnull().sum())

# Impute missing values using median
imputer = SimpleImputer(strategy="median")
df[columns] = imputer.fit_transform(df[columns])

print("\nMissing Values After Imputation:")
print(df.isnull().sum())
```

## Correlation Matrix & Heatmap

```
corr = df.corr(numeric_only=True)

print("\nCorrelation Matrix:")
print(corr)

plt.figure(figsize=(10, 8))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```

## Distance Calculations (Original Scale)

```
# Select sample data for distance calculation
if "outcome" in df.columns:
    sample = df.drop("outcome", axis=1).iloc[0:2]
else:
    sample = df.iloc[0:2]

print("\nSample Used for Distance:")
print(sample)

print("\nEuclidean Distance:")
print(euclidean_distances(sample))

print("\nManhattan Distance:")
print(manhattan_distances(sample))
```

```
print("\nC cosine Distance:")
print(cosine_distances(sample))
```

## Standardization

```
# Feature selection for standardization
if "outcome" in df.columns:
    features = df.drop("outcome", axis=1)
else:
    features = df.copy()

# Standardization
scaler = StandardScaler()
scaled = scaler.fit_transform(features)

scaled_df = pd.DataFrame(scaled, columns=features.columns)

print("\nStandardized Data:")
print(scaled_df.head())
```

## Distance Calculations (After Scaling)

```
scaled_sample = scaled_df.iloc[0:2]

print("\nEuclidean Distance After Scaling:")
print(euclidean_distances(scaled_sample))

print("\nManhattan Distance After Scaling:")
print(manhattan_distances(scaled_sample))

print("\nC cosine Distance After Scaling:")
print(cosine_distances(scaled_sample))
```

## Visualizations

```
# Compare original and standardized Glucose distributions
if "glucose" in features.columns:
    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.hist(features["glucose"], bins=20)
    plt.title("Original Glucose")

    plt.subplot(1, 2, 2)
    plt.hist(scaled_df["glucose"], bins=20)
    plt.title("Standardized Glucose")

    plt.tight_layout()
    plt.show()
```

```
# Scatter plot: Glucose vs BMI
if "glucose" in features.columns and "bmi" in features.columns:
    plt.figure(figsize=(7, 5))
    sns.scatterplot(
        data=df,
        x="glucose",
        y="bmi"
    )

    plt.title("Glucose vs BMI")
    plt.xlabel("Glucose")
    plt.ylabel("BMI")
    plt.show()

# Scatter plot: Age vs Glucose
if "age" in features.columns and "glucose" in features.columns:
    plt.figure(figsize=(7, 5))
    sns.scatterplot(
        data=df,
        x="age",
        y="glucose"
    )
    plt.title("Age vs Glucose")
    plt.xlabel("Age")
    plt.ylabel("Glucose")
    plt.show()
```